

**Algores - Analyse critique : *Formalizing Rollback Netcodes for
Robust and Real-Time Client-Server Architectures***

HARRAOUI Inès 21204796

HUANG Loïc 21203938

1 Introduction

L'industrie du jeu vidéo a énormément évolué ces dernières années, passant de jeux hors ligne simples à des jeux multijoueurs en ligne de plus en plus complexes et immersifs. Parmi ces évolutions, la gestion de la latence réseau est devenue un pilier essentiel du développement des jeux en ligne. Dans l'article *Formalizing Rollback Netcodes for Robust and Real-Time Client-Server Architectures* écrit par Yérom-David Bromberg, Jérémie Decouchant, Manon Sourisseau et François Taïani, les auteurs s'intéressent au problème suivant : comment faire en sorte que le jeu reste fluide et réactif, malgré la latence réseau et les attaques possibles sur le réseau. Pour cela, ils s'intéressent au netcode, et plus particulièrement au rollback netcode, qui est utilisé dans la grande majorité des jeux en ligne.

L'article propose une formalisation théorique du rollback netcode et essaie de savoir sous quelles conditions on peut avoir à la fois une bonne immersion (jeu fluide) et une certaine robustesse face à des attaques.

2 Résumé structuré de l'article

Dans un premier temps, les auteurs présentent les différentes notions abordées, ensuite ils modélisent le mécanisme du rollback netcode dans une architecture client-serveur afin de mieux visualiser leurs limites et leurs problèmes.

2.1 Netcode et latence réseau

Le netcode est la partie du code source d'un jeu en ligne qui gère la communication entre les clients (le logiciel du jeu) et le serveur, ainsi que la synchronisation de l'état du jeu entre tous les joueurs. Dans un jeu en ligne, chaque joueur envoie ses actions à un serveur, dont le rôle est de s'assurer que tous les joueurs voient le même état du jeu au même moment. Le problème principal vient du fait que les messages mettent un certain temps à traverser le réseau (la latence) et que celle-ci peut être différente selon les joueurs et le réseau de chacun. Pour être plus précis, chaque joueur envoie ses actions, par exemple : tirer, sauter etc.. au serveur et le serveur simule l'état du jeu depuis les actions envoyées par les joueurs. Cet état est ensuite renvoyé aux clients. Imaginons que le logiciel du jeu attend toujours la réponse du serveur pour afficher les actions du joueur, l'expérience en jeu peut être très désagréable en fonction de la latence du joueur.

2.2 Principe du rollback netcode

L'idée du rollback netcode est alors de laisser le client afficher immédiatement l'action du joueur, sans attendre la réponse du serveur. Lorsque le client reçoit la réponse du serveur, il vérifie si l'état correspond à ce que le client avait simulé localement. Si ce n'est pas le cas, le client exécute alors un rollback, c'est-à-dire qu'il revient à un état précédent, qui sera cohérent avec l'état du serveur, puis rejouera toutes les actions que le joueur avait fait pendant l'attente de la réponse du serveur. Cela permet d'améliorer la fluidité du jeu en affichant immédiatement les actions du joueur. Cependant, elle rend la gestion du réseau plus complexe et crée des désynchronisations entre les clients et le serveur.

Dans l'article, les auteurs insistent sur le fait que même si cette technique est très utilisée dans les jeux vidéo, il y a que très peu d'articles scientifiques qui parlent de sa sécurité et des vulnérabilités qu'il peut engendrer.

2.3 Architecture Client-Serveur

Dans cette architecture, chaque joueur exécute un client et envoie ses entrées au serveur. Les auteurs modélisent le jeu comme un système de ticks, des unités de temps où le jeu met à jour son état en prenant en compte les actions des joueurs, les calculs de la physique et du comportement de l'IA à une fréquence fixe. Le serveur simule le jeu avec un petit retard par rapport aux clients, afin de prendre en compte le temps de transmission de la part des clients. De plus, pour éviter que le serveur soit bloqué à cause d'un message non reçu de la part d'un client, le serveur dispose d'un délai maximum d'attente. Si certaines entrées n'arrivent pas à temps, il essaye de les prédire et continue la simulation. Ces prédictions sont traitées comme étant l'action du joueur même si celui-ci ne l'a pas fait. Le rollback netcode provoque alors une désynchronisation permanente entre les clients car chacun voit ses propres actions en avance et celles des autres joueurs avec un certain retard. Chacun a alors un état de jeu différent des autres.

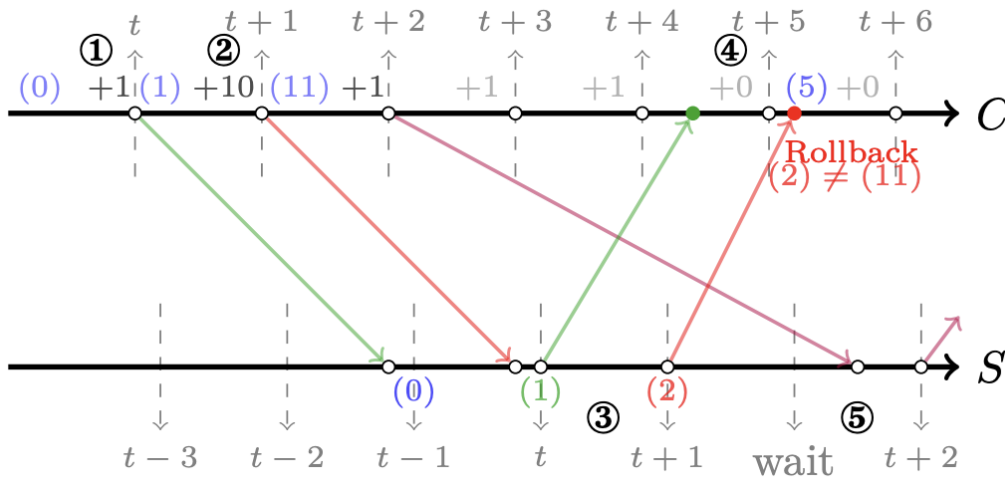


FIGURE 1 – Aperçu de la transmission des messages dans un réseau rollback, entre un client et le serveur extrait de l'article (Source : Formalizing Rollback Netcodes for Robust and Real-Time Client-Server Architectures, OPODIS 2025)

2.4 Attaques de latence

L'article révèle également que la désynchronisation du au rollback netcode de l'architecture client-serveur, peut être exploitée et amplifiée par des joueurs malveillants. Les auteurs se concentrent sur deux types d'attaques de latence courantes dans les jeux en ligne : le Lag-switch et les attaques par déni de service distribué (DDoS).

- **Lag-switch** : Le principe du lag-switch est de couper temporairement ou ralentir sa propre connexion internet via un matériel ou logiciel, afin de provoquer un retard artificiel dans l'envoi des messages au serveur. Pendant cette période, l'attaquant peut effectuer des actions localement sans que les autres joueurs puissent réagir. Lorsque la connexion est rétablie et que les paquets en retard sont transmis au serveur, le serveur les accepte, ce qui déclenche des rollbacks chez les autres joueurs et lui donne un avantage.
- **DDoS** : Le principe d'une attaque par déni de service, est de surcharger la connexion réseau d'un autre joueur, en lui envoyant une grande quantité de trafic à partir de bots. La latence créée par l'attaque dégrade la connexion entre le joueur attaqué et le serveur, ce qui provoque un retard important dans la réception des messages du serveur ou la transmission de celui-ci donc il subit de nombreux rollbacks.

Dans les deux types d'attaques, le serveur ne peut pas distinguer si cette latence a été provoqué par des comportements malveillants où venant réellement du joueur. Tant que les messages finissent par arriver avant le délai maximal d'attente, ils sont intégrés dans la simulation du serveur comme s'il s'agissait d'un retard "normal", ce qui rend ces attaques difficiles à détecter.

2.5 Propriétés

L'article définit ensuite plusieurs propriétés mathématiques afin de modéliser le fonctionnement du rollback netcode.

- **Propriété 1 : Validité côté client** : L'état d'un client au tick t doit être obtenu à partir du dernier état reçu du serveur, puis en appliquant la liste des actions effectuées par le joueur jusqu'à cet instant. Par exemple, si l'état du client est au 110^e tick et que le dernier état reçu du serveur correspond au 100^e tick, le client repart de l'état au 100^e tick et rejoue toutes les actions réalisées entre les ticks 100 et 110. Cette propriété modélise le fonctionnement du rollback où le client repart de l'état envoyé par le serveur et re-simule les entrées qu'il avait déjà jouées.
- **Propriété 2 : Validité côté serveur** : L'état du serveur à un tick donné est calculé à partir de l'état précédent et des actions reçues depuis les différents joueurs au même tick. De plus, l'état du serveur joue un rôle important en tant que référence lorsque les clients ont des états différents.
- **Propriété 3 : Immersion et cohérence** : Pour définir ce qu'est un jeu fluide et cohérent du point de vue d'un joueur, les auteurs définissent la notion de distance sémantique entre les états du jeu, dans lequel chaque action/paramètre est caractérisé par un poids. Lorsque la distance sémantique de l'état du serveur et l'état du client est proche, on dit que l'immersion est garantie. Dans l'article, les auteurs prennent l'exemple d'un jeu de combat, une petite différence de position entre deux joueurs peut avoir un faible poids alors que l'activation d'une attaque spéciale peut avoir un poids plus important car cela impacte beaucoup plus l'expérience du joueur.

Les auteurs parlent également d'immersion idéale, lorsque le joueur ne voit jamais de rollback, ou alors que les corrections sont tellement petites qu'elles ne gâchent pas l'expérience de jeu.

3 Points positifs de l'article

3.1 Sujet très peu étudié

L'article est très pertinent du fait qu'il n'existe que très peu d'articles qui évoquent le mécanisme du rollback netcode orienté sécurité, en évoquant les vulnérabilités qu'il engendre. Alors que dans l'industrie du jeu vidéo, la plupart des jeux en ligne utilisent ce mécanisme. Les auteurs l'affirment également (cf. Section "Abstract"). Nos recherches ont également mené à cette même conclusion.

3.2 Simplicité

Pour mieux expliquer certaines notions, les auteurs s'appuient sur des exemples comme les jeux de tir ou les jeux de combat, ce qui facilite la compréhension. De plus, l'article est très bien structuré et rédigé de manière simple.

4 Limites et critiques

Dans cette section, nous présentons les limites de l'article.

4.1 Modèle de réseau simplifié

Les auteurs se concentrent principalement sur la latence. Or, dans un réseau, il est possible que les messages envoyés soient perdus et que les paquets n'arrivent pas dans le même ordre dans lesquels ils ont été émis, etc... Dans l'article, ces concepts n'ont pas été pris en compte. Si ces concepts étaient ajoutés dans le modèle, les résultats auraient-ils été différents ? Ce n'est pas très réaliste.

4.2 Définition subjective de l'immersion

Les auteurs définissent l'immersion via une distance sémantique selon l'importance des actions ou événements dans le jeu. Cependant, en fonction du type de jeu ou des personnes, le poids des actions ou événements peuvent être différents. Cette différence ne permet pas d'avoir une notion d'immersion qui soit universelle pour tous. Par exemple, est-ce que deux jeux très différents (un jeu de combat et un jeu coopératif) peuvent être comparés avec la même notion d'immersion ? Un est basé sur la réactivité, l'autre est plutôt basé dans l'expérience proposée.

4.3 Vision limitée sur la sécurité

L'article se focalise principalement sur des attaques de latence et montre qu'elles peuvent casser l'immersion du joueur. C'est cohérent avec les objectifs de l'article, puisque ce sont des vulnérabilités liées directement au rollback netcode. Mais, dans un jeu en ligne, il existe beaucoup d'autres types de triches comme la modification du client, aimbots, etc.. qui affectent également l'expérience du joueur. A la fin, il semble qu'il n'y a que les vulnérabilités liées au rollback qui gâchent l'expérience de jeu.

4.4 Article très théorique

L'article est très théorique, les auteurs définissent un modèle avec des propriétés, et prouve des propositions de manière théorique. Cependant, ils ne montrent pas d'implémentation réelle, d'expérience utilisateur ou de prototype d'implémentation. Nous ne savons pas si toutes les propositions prouvées théoriquement par les auteurs sont en réalité valides.

4.5 Portée limitée

L'article se concentre sur l'architecture de type Client-Serveur. Cependant, même si l'architecture Peer-to-Peer a été évoquée dans l'article, les auteurs n'ont fait qu'une brève comparaison des deux architectures pour expliquer leur choix de se concentrer sur Client-Server (cf Section "Client-Server Architecture and Netcodes"). Or, ils auraient pu faire une comparaison des deux architectures orientée sur la sécurité : il est possible que les vulnérabilités en Peer-to-peer ne soient pas les mêmes qu'en Client-Server. De plus, l'article se concentre sur le rollback netcode, mais ne parle pas beaucoup des autres approches de netcode, comme le delay-based netcode. Selon nous, une comparaison des vulnérabilités des netcodes en fonction d'une architecture aurait pu améliorer le contenu de l'article.

5 Conclusion

Pour conclure, l'article *Formalizing Rollback Netcodes for Robust and Real-Time Client-Server Architectures* parle d'un sujet très peu étudié dans la recherche, malgré qu'il soit très important dans l'industrie du jeu vidéo. Les auteurs proposent une vision théorique du rollback netcode en le formalisant et en analysant ses vulnérabilités face aux attaques de latence. L'article permet de mieux comprendre les limites de l'architecture client-serveur des jeux en ligne, notamment où la latence peut être manipulée par des joueurs malveillants.

Cependant, l'article comporte plusieurs limites menant à des résultats peu réalistes à cause des conditions trop éloignées de la réalité. Le modèle réseau proposé est très simplifié et ne prend pas en compte certaines contraintes telles que la perte de paquets. De plus, l'article ne proposant pas d'implémentation ni d'expérience utilisateur, ne nous permet pas de savoir si le modèle présenté fonctionne dans des conditions réelles.

Malgré ces limites, nous pouvons conclure que cet article reste une bonne introduction théorique au fonctionnement du rollback netcode.